Linux Commands for DevOps: A Simple Guide

In DevOps, Linux is the foundation for servers, cloud virtual machines, Docker containers, and CI/CD pipelines. This guide covers the essential Linux commands you will use daily, explained in plain, everyday language with real-world DevOps examples.

1. File & Directory Navigation (Moving Around & Managing Files)

Think of these commands as how you open folders, create files, and move things around when you don't have a mouse or graphical screen.

Command	Simple Explanation	Common DevOps Example
`pwd`	**Print Working Directory**: Shows the exact folder path you are currently standing in.	`pwd` (Check where you are after logging into a server via SSH)
`ls`	**List**: Shows all files and folders inside your current folder.	`ls -la` (Lists everything, including hidden `.env` or `.git` files, with permissions and sizes)
`cd`	**Change Directory**: Moves you into a different folder.	`cd /var/log/nginx` (Go to the Nginx logs folder), `cd ..` (Go back one folder)
`mkdir`	**Make Directory**: Creates a brand new folder.	`mkdir -p /opt/app/config` (`-p` creates parent folders automatically if they don't exist)
`touch`	Creates an empty file (or updates the timestamp of an existing file).	`touch Dockerfile` (Quickly create a blank Dockerfile)
`cp`	**Copy**: Copies a file or folder to another location.	`cp config.yaml config.yaml.bak` (Make a quick backup before editing a config file), `cp -r src/ backup/` (`-r` copies entire folders)
`mv`	**Move / Rename**: Moves a file/folder to a new place, or renames it.	`mv app.jar /opt/app/` (Move build artifact), `mv old.conf new.conf` (Rename a file)
`rm`	**Remove**: Permanently deletes files or folders (there is no Recycle Bin!).	`rm temp.txt` (Delete a file), `rm -rf build/` (`-r` deletes folders, `-f` forces it without asking—use carefully!)
`find`	Searches for files or folders by name, size, or age.	`find /var/log -name "*.log" -mtime +7` (Find all `.log` files older than 7 days to clean up disk space)

Command	Simple Explanation	Common DevOps Example
`ln`	**Link**: Creates a shortcut (symlink) pointing to another file or folder.	`ln -s /etc/nginx/sites-available/app /etc/nginx/sites-enabled/app` (Enable an Nginx site configuration)

2. Viewing, Searching & Editing Text (Log Analysis & Configs)

DevOps engineers spend a lot of time reading application logs and updating configuration files (`.yaml`, `.json`, `.conf`, `.env`).

Command	Simple Explanation	Common DevOps Example
`cat`	**Concatenate**: Prints the entire content of a file directly onto your screen. Best for short files.	`cat /etc/os-release` (Check which Linux version the server is running)
`less`	Opens a large file page-by-page so it doesn't flood your screen (press `q` to exit, `/` to search).	`less /var/log/syslog` (Read through a massive system log file comfortably)
`head`	Shows only the **first** few lines of a file (10 lines by default).	`head -n 20 app.log` (See the first 20 lines of a log file)
`tail`	Shows only the **last** few lines of a file. Using `-f` lets you watch new lines appear live in real time!	`tail -f /var/log/nginx/error.log` (Watch errors live while testing a website deployment)
`grep`	Searches inside files or command output for a specific word or pattern (like Ctrl+F).	`grep -i "error" app.log` (`-i` ignores uppercase/lowercase), `grep -r "DB_HOST" /etc/app/` (Search a whole folder)
`awk`	Pulls out specific columns from text output. Great for extracting a single piece of data.	`awk '{print $1}' access.log` (Print only the 1st column—such as visitor IP addresses—from a web server log)
`sed`	**Stream Editor**: Finds and replaces text inside a file without even opening it.	`sed -i 's/localhost/10.0.1.5/g' config.env` (Replace `localhost` with `10.0.1.5` directly inside `config.env`)
`wc`	**Word Count**: Counts lines, words, or characters.	`grep "500" access.log | wc -l` (`-l` counts how many lines matched—e.g., how many 500 server errors occurred)
`sort` & `uniq`	Sorts lines alphabetically/numerically and removes or counts duplicates.	`awk '{print $1}' access.log | sort | uniq -c | sort -nr` (Find which IP addresses hit your server the most)

Command	Simple Explanation	Common DevOps Example
`jq`	Formats, colors, and extracts data from JSON responses (essential for cloud CLIs and APIs).	**`curl -s api.example.com/status | jq '.health'`** (Pull just the **`health`** field from a JSON API response)
`nano` / **`vim`**	Terminal text editors used to edit files directly on a server. (**`nano`** is simple; **`vim`** is powerful).	**`nano /etc/nginx/nginx.conf`** (Open and edit a config file; in **`vim`**, press **`i`** to type, **`Esc`** then **`:wq`** to save and quit)

3. Permissions & Ownership (Access Control & Security)

Linux is strict about who can read, edit, or run a file. Many "Permission Denied" errors in CI/CD pipelines or Docker containers are fixed with these commands.

Quick Concept — The 3 Permission Numbers:

- **4 = Read (`r`)**: Allowed to view the file.
- **2 = Write (`w`)**: Allowed to edit or delete the file.
- **1 = Execute (`x`)**: Allowed to run the file as a program/script.

*Add them together: **`7`** = 4+2+1 (Read, Write, Execute), **`6`** = 4+2 (Read, Write), **`5`** = 4+1 (Read, Execute), **`0`** = No access. Three digits like **`755`** set permissions for **Owner**, **Group**, and **Everyone Else**.*

Command	Simple Explanation	Common DevOps Example
`chmod`	**Change Mode**: Changes who can read, write, or execute a file.	**`chmod +x deploy.sh`** (Make a script runnable), **`chmod 400 key.pem`** (Lock down an SSH private key so only you can read it)
`chown`	**Change Owner**: Changes which user and group own a file or folder.	**`chown -R www-data:www-data /var/www/html`** (Give the web server user ownership of website files)
`sudo`	**Superuser Do**: Runs a single command with full administrator (root) privileges.	**`sudo systemctl restart nginx`** (Restart a system service that regular users aren't allowed to touch)
`su`	**Switch User**: Switches your terminal session to another user account.	**`sudo su - jenkins`** (Switch into the **`jenkins`** user to test why a pipeline command is failing)
`useradd` / **`usermod`**	Creates a new user account or modifies an existing user's settings/groups.	**`sudo usermod -aG docker ubuntu`** (Add the **`ubuntu`** user to the **`docker`** group so they can run Docker without **`sudo`**)
`whoami` & **`id`**	Shows which user you are currently logged in as and which groups you belong to.	**`id`** (Verify your current user ID and group memberships)

4. Process & Service Management (Running & Stopping Apps)

When a server is slow, an app crashes, or you deploy new code, you need to check what programs (processes) and background services are running.

Command	Simple Explanation	Common DevOps Example
`ps`	**Process Status**: Takes a snapshot of currently running programs.	`ps aux | grep python` (Check if your Python app is currently running and see its Process ID / PID)
`top` / `htop`	Live Task Manager for Linux. Shows real-time CPU and RAM usage per process (`q` to quit).	`htop` (Quickly spot which runaway process is eating 100% CPU)
`kill`	Stops a stuck or unwanted program using its Process ID (PID).	`kill 1234` (Politely ask PID 1234 to shut down), `kill -9 1234` (`-9` forces an immediate kill if it's frozen)
`pkill` / `killall`	Stops a program by its **name** instead of looking up its PID number.	`pkill -f node` (Stop running Node.js processes)
`systemctl`	Starts, stops, restarts, and checks background services (like Nginx, Docker, SSH).	`sudo systemctl status docker` (Check if Docker is running), `sudo systemctl restart nginx` (Restart Nginx), `sudo systemctl enable nginx` (Start on boot)
`journalctl`	Reads logs generated by background `systemd` services.	`journalctl -u nginx -f` (Follow live logs specifically for the `nginx` service)
`nohup` & `&`	Runs a command in the background so it keeps running even if you close your terminal.	`nohup ./start-server.sh &` (Run a script in the background and save output to `nohup.out`)
`crontab`	Schedules scripts or commands to run automatically at fixed times (like a recurring alarm).	`crontab -e` (Edit scheduled jobs—e.g., `0 2 * * * /opt/backup.sh` runs a backup every night at 2:00 AM)

5. System Monitoring & Disk Space (Health & Troubleshooting)

When an alert fires at 2 AM saying a production server is unhealthy or out of space, these are the first commands you run.

Command	Simple Explanation	Common DevOps Example
`df`	**Disk Free**: Shows how much storage space is used and available on your hard drives.	`df -h` (`-h` makes sizes "human-readable" in GB/MB instead of raw bytes—crucial when a disk hits 100%)
`du`	**Disk Usage**: Shows how much space specific folders or files are taking up.	`du -sh /var/log/*` (See the total size of each item inside `/var/log` to find what filled up the disk)

Command	Simple Explanation	Common DevOps Example
`free`	Shows how much RAM (memory) and Swap space is free, used, and cached.	**`free -h`** (Check available memory in GB/MB to see if the server is running out of RAM)
`uptime`	Shows how long the server has been running since its last reboot and the average CPU load.	**`uptime`** (Quickly check if the server recently rebooted or is overloaded)
`lscpu`	Displays details about the server's CPU (number of cores, architecture).	**`lscpu`** (Verify how many vCPUs your cloud VM actually has)
`uname`	Shows system and Linux kernel information.	**`uname -a`** (View kernel version and system architecture, e.g., **`x86_64`** vs **`arm64`**)
`dmesg`	Prints kernel hardware and system events—especially Out-Of-Memory (OOM) kills.	**`dmesg -T | grep -i "killed process"`** (Check if Linux automatically killed your app/container because it ran out of RAM)
`lsof`	**List Open Files**: Shows which files or network ports are currently held open by which programs.	**`sudo lsof -i :8080`** (Find out which process is already using port 8080 when your app fails with "Address already in use")

6. Networking & Connectivity (Ports, DNS, Firewalls & APIs)

In DevOps, services constantly talk to each other over networks. These commands help you debug connection timeouts, DNS issues, and blocked ports.

Command	Simple Explanation	Common DevOps Example
`ip` / **`ifconfig`**	Shows your server's IP addresses and network interfaces.	**`ip a`** (Find your server's private IP address)
`ping`	Sends a tiny test packet to another server to see if it is reachable and how fast it replies.	**`ping -c 4 google.com`** (Send 4 test packets to check internet connectivity)
`curl`	Makes web requests (HTTP/HTTPS) from the terminal—like a web browser without graphics.	**`curl -I https://example.com`** (**`-I`** fetches only HTTP headers/status code like **`200 OK`**), **`curl -X POST ...`** (Test an API endpoint)
`wget`	Downloads files directly from a web URL onto your server.	**`wget https://example.com/app.tar.gz`** (Download an installer or release archive)
`ss` / **`netstat`**	Lists all open network ports and active connections on the server (**`ss`** is the modern, faster version).	**`sudo ss -tulpn`** (See all listening TCP/UDP ports and the exact programs using them)

Command	Simple Explanation	Common DevOps Example
`dig` / **`nslookup`**	Checks DNS records to see which IP address a domain name points to.	**`dig api.example.com +short`** (Verify if your DNS change has updated to the new IP address)
`nc` (netcat)	Tests whether a specific port on a remote server is open and reachable.	**`nc -zv db.internal 5432`** (Check if your app server can reach the database server on port 5432)
`traceroute`	Shows every router ("hop") your traffic passes through to reach a destination.	**`traceroute 8.8.8.8`** (See where network packets are getting stuck or delayed)
`ssh`	**Secure Shell**: Securely logs you into a remote Linux server's terminal.	**`ssh -i key.pem ubuntu@203.0.113.10`** (Connect to an AWS EC2 instance using a private key)
`scp` / **`rsync`**	Copies files securely between your computer and a remote server over SSH (**`rsync`** only copies changes, making it much faster).	**`scp app.zip ubuntu@10.0.1.5:/tmp/`** (Upload **`app.zip`**), **`rsync -avz ./dist/ user@server:/var/www/app/`** (Sync a deployment folder)
`ufw` / **`iptables`**	Manages the Linux firewall to allow or block network ports.	**`sudo ufw allow 443/tcp`** (Open HTTPS port 443 in the firewall)

7. Archives, Compression & Package Management (Installing & Bundling)

Whether you are installing tools inside a Dockerfile or packaging build artifacts in CI/CD, you'll use these regularly.

Command	Simple Explanation	Common DevOps Example
`tar`	Packs multiple files into a single **`.tar`** or **`.tar.gz`** archive, or unpacks them.	**`tar -czvf backup.tar.gz /etc/nginx`** (**c**reate **z**ipped archive), **`tar -xzvf backup.tar.gz`** (e**x**tract **z**ipped archive)
`zip` / **`unzip`**	Compresses files into a standard **`.zip`** file or extracts them.	**`unzip artifact.zip -d /opt/app/`** (Extract a **`.zip`** file into **`/opt/app/`**)
`apt` / **`apt-get`**	The App Store (package manager) for **Ubuntu / Debian** Linux.	**`sudo apt update && sudo apt install -y nginx curl`** (Refresh package list and install Nginx and curl without prompts)
`yum` / **`dnf`**	The App Store (package manager) for **RedHat / CentOS / Amazon Linux**.	**`sudo dnf install -y git docker`** (Install Git and Docker on Amazon Linux / RHEL)

Command	Simple Explanation	Common DevOps Example
apk	The lightweight package manager for **Alpine Linux** (very common in tiny Docker images).	**`apk add --no-cache bash curl`** (Install bash and curl inside an Alpine Docker container)

8. Environment Variables & Shell Operators (Connecting Commands Together)

In DevOps, apps read configuration (like database URLs or API keys) from environment variables, and engineers chain small commands together into powerful one-liners.

Environment & Shell Commands

- **`env` / `printenv`**: Lists all currently active environment variables on the system.
- **`export`**: Creates an environment variable for your current terminal session and any app you start from it (e.g., **`export APP_ENV=production`**).
- **`echo`**: Prints text or the value of a variable to the screen (e.g., **`echo $APP_ENV`** or **`echo $PATH`**).
- **`history`**: Shows a numbered list of commands you ran previously (Tip: press **`Ctrl + R`** in your terminal to search your command history interactively!).
- **`alias`**: Creates a short nickname for a long command (e.g., **`alias k='kubectl'`**).

Essential Shell Operators (Pipes & Redirects)

- **`|` (Pipe)**: Takes the output of the command on the left and feeds it as input into the command on the right. Example: **`ps aux | grep nginx`** (List all processes, then filter only lines containing "nginx").
- **`>` (Overwrite Redirect)**: Saves command output into a file, **overwriting** the file if it already exists. Example: **`echo "server_name localhost;" > site.conf`**
- **`>>` (Append Redirect)**: Adds command output to the **bottom** of an existing file without erasing what's already there. Example: **`echo "10.0.0.5 db-server" >> /etc/hosts`**
- **`&&` (AND Operator)**: Runs the second command **only if** the first command succeeded (exit code **`0`**). Example: **`mkdir build && cd build`**
- **`$?` (Last Exit Code)**: Holds the status code of the last command that ran (**`0`** means success; any non-zero number like **`1`** or **`127`** means an error occurred—critical for CI/CD scripts!).